

Task: Build and Deploy a Docker Application Using Jenkins and Amazon ECR

Objective

Design and implement a Jenkins CI/CD pipeline that automates the process of building a Docker image, pushing it to Amazon Elastic Container Registry (ECR), pulling the image from ECR, and deploying it as a Docker container on the same Jenkins server.

Scenario

Your team is responsible for automating the deployment of a containerized application. The application source code is stored in a Git repository, and Amazon ECR will be used as the Docker image registry.

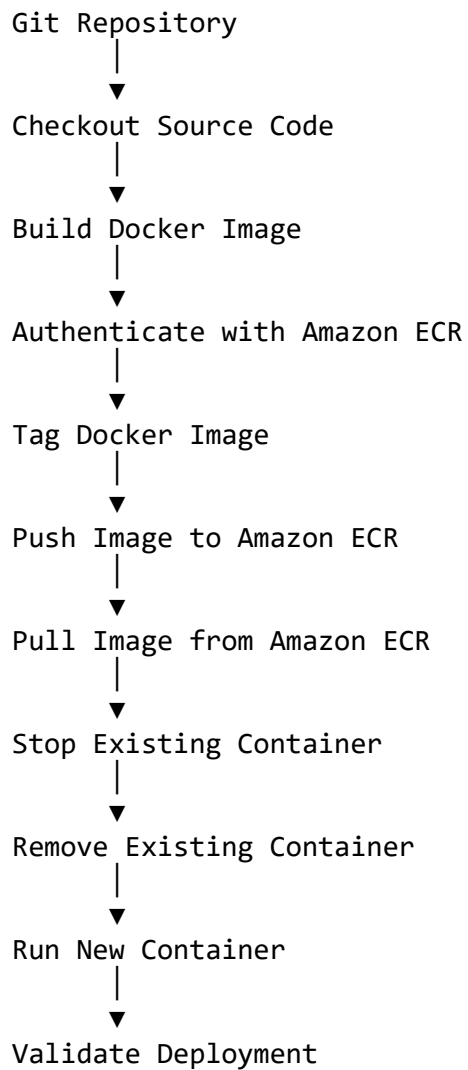
The Jenkins pipeline should perform the complete deployment workflow from source code checkout to container deployment.

Requirements

The Jenkins pipeline must perform the following tasks in sequence:

1. Check out the application source code from the configured Git repository.
 2. Build a Docker image using the application's Dockerfile.
 3. Authenticate with Amazon ECR using appropriate AWS credentials.
 4. Tag the Docker image with a unique version (for example, build number or commit ID).
 5. Push the Docker image to the designated Amazon ECR repository.
 6. Pull the same Docker image from Amazon ECR.
 7. Stop and remove any existing container running the application (if present).
 8. Create and start a new Docker container using the image pulled from ECR.
 9. Expose the application on the required host port.
 10. Verify that the container is running successfully.
 11. Display the deployment status at the end of the pipeline execution.
-

Expected Pipeline Flow



Deliverables

Submit the following:

- Jenkins Pipeline (Jenkinsfile)
- Dockerfile
- Screenshots or logs showing:
 - Successful Docker image build
 - Successful push to Amazon ECR
 - Successful image pull from Amazon ECR
 - Running Docker container
 - Application accessible through the configured port

Validation Criteria

The submission will be evaluated based on the following:

- Successful source code checkout
 - Docker image built without errors
 - Successful authentication with Amazon ECR
 - Docker image pushed successfully to ECR
 - Docker image pulled successfully from ECR
 - Existing container handled gracefully
 - New container created successfully
 - Application is accessible after deployment
 - Pipeline completes successfully without manual intervention
-

Assumptions

- Jenkins, Docker, and AWS CLI are installed on the Jenkins server.
 - Docker service is running.
 - An Amazon ECR repository has already been created.
 - Appropriate AWS IAM permissions are available for ECR operations.
 - The application includes a valid Dockerfile.
-

Bonus Tasks (Optional)

- Implement image versioning using Git commit hash.
 - Add Docker image cleanup after deployment.
 - Perform application health checks after deployment.
 - Add rollback capability if deployment fails.
 - Archive build logs and deployment reports.
 - Send deployment notifications (Email, Slack, or Microsoft Teams).
 - Parameterize the pipeline for different environments (Development, QA, Production).
-

Expected Outcome

Upon successful completion, the Jenkins pipeline should automatically:

- Build a Docker image from the application source code.

- Push the image to Amazon ECR.
- Pull the image back from Amazon ECR.
- Deploy the latest image as a Docker container on the Jenkins server.
- Ensure the application is running and accessible through the configured port.